

Documentación Técnica y de Tecnología

Súper CLABE

Webapp de Cobros y Pagos SPEI / CoDi — capa no-custodia

Versión	1.0
Fecha	Julio 2026
Ecosistemas	SPEI · CoDi · Dimo · Ley Fintech
Clasificación	Confidencial — uso interno de desarrollo

Sistema: Súper CLABE - Webapp de Cobros y Pagos SPEI / CoDi (capa no-custodia)

Versión: 1.0

Fecha: Julio 2026

Clasificación: Confidencial - uso interno de desarrollo

1. Resumen ejecutivo

Súper CLABE es una plataforma web de cobranza y enrutamiento de transferencias que opera como **capa de software no-custodia** sobre la infraestructura del Banco de México (SPEI, CoDi y Dimo). El dinero viaja directamente cuenta-a-cuenta entre los bancos de los usuarios; la plataforma **nunca capta ni retiene fondos**, evitando la clasificación como Institución de Fondos de Pago Electrónico (IFPE) y su capital mínimo de 5 millones de UDIS. La única deducción es una tarifa de licenciamiento de software del **0.05%** debitada de un saldo de membresía prepago del emisor.

El sistema implementa seis módulos regulatorios: onboarding con KYC gradual, bóveda bancaria cifrada, generación de cobros CoDi mediante código QR, pago/checkout con doble factor, membresía prepago y bitácoras de auditoría inmutables.

Este documento describe la arquitectura, el stack tecnológico, el modelo de datos, la API, los controles de seguridad, el frontend, los flujos operativos y el despliegue en AWS EC2.

2. Modelo regulatorio y de negocio

Aspecto	Definición
Figura regulatoria	Proveedor de Servicios de Participación Indirecta (PSPI) / Iniciador de Pagos, patrocinado por un participante directo (banco o IFPE).
Marco normativo	Circular 14/2017 (Reglas del SPEI), Circular 12/2019 (CoDi), Ley Fintech, LTOSF, Guías de homologación UX (Circular 9/2026).
No-custodia	No hay cuentas de depósito, wallets ni crédito. Solo enrutamiento e información.
Monetización	Membresía prepago; se debita 0.05% al emisor por transacción liquidada. Prohibido cobrar comisión al receptor CoDi.
Bloqueo lógico	Si el saldo de membresía llega a cero, se impide generar/procesar nuevas operaciones hasta recargar.

3. Arquitectura del sistema

3.1 Vista de despliegue (producción, EC2)

```

Internet :80/:443
----->
+----- EC2 (Ubuntu 22.04) -----+
|
|   Nginx (reverse-proxy)
|   +-- /          -> frontend estático (SPA index.html)
|   +-- /api/*     -> backend
|   +-- /docs      -> backend (Swagger)
|
|   backend: FastAPI + Gunicorn (workers Uvicorn) :8000
|               +--> PostgreSQL 16 (volumen)
|               +--> Redis 7.2 (sesiones / rate-limit)
|
+-----+

```

Todos los componentes corren como contenedores orquestados por `docker-compose.prod.yml`.

Solo el puerto web queda expuesto; la base de datos y la API no se publican directamente.

3.2 Estilo arquitectónico

- **Backend por capas:** enrutadores (HTTP) -> esquemas (validación) -> modelos (ORM) -> utilidades transversales (cifrado, seguridad, auditoría).
- **Frontend SPA:** una sola página con módulos conmutados por JavaScript; consume la API del mismo origen a través de Nginx (o funciona en modo demo local con persistencia en el navegador).
- **Separación de responsabilidades:** Nginx (TLS/estático/proxy), Gunicorn (proceso ASGI), FastAPI (lógica), PostgreSQL (persistencia), Redis (caché/sesiones/rate-limit).

4. Stack tecnológico

4.1 Backend

Tecnología	Versión	Rol / justificación
Python	3.12	Lenguaje base del backend.
FastAPI	0.115.0	Framework web asíncrono; OpenAPI/Swagger automático, validación por tipos.
Uvicorn	0.30.6	Servidor ASGI (desarrollo con recarga en caliente).
Gunicorn	22.0.0	Gestor de procesos en producción con workers UvicornWorker.
SQLAlchemy	2.0.35	ORM y capa de acceso a datos (estilo declarativo 2.0).
Pydantic	2.9.2	Validación de contratos de entrada/salida (esquemas).
psycopg2-binary	2.9.9	Driver PostgreSQL.
python-jose	3.3.0	Firma y verificación de JWT de sesión.
cryptography	43.0.1	Cifrado simétrico AES-256 de datos sensibles.

Tecnología	Versión	Rol / justificación
PostgreSQL	16	Base de datos relacional (SQLite como fallback de desarrollo).
Redis	7.2	Caché en memoria, control de sesiones y rate-limiting.

4.2 Frontend

Tecnología	Rol
HTML5 + CSS3 + JavaScript (ES2020)	SPA de una sola página, sin framework, sin dependencias de build.
qrcodejs (CDN)	Generación de códigos QR de los Mensajes de Cobro CoDi.
jsQR (CDN)	Lectura/decodificación de QR desde cámara o imagen.
Web Crypto API (crypto.subtle)	Cálculo de SHA-256 para la cadena de auditoría en el cliente.
localStorage	Persistencia de datos de prueba entre recargas.

Los CDN usan cdnjs con respaldo automático a jsDelivr; si ambos fallan, la app degrada con elegancia (muestra el contenido del QR y permite pegar/simular el escaneo).

4.3 Infraestructura

Tecnología	Rol
Docker + Docker Compose	Empaquetado y orquestación (dev y prod).
Nginx (alpine)	Reverse-proxy, servidor estático, terminación TLS.
AWS EC2 (Ubuntu 22.04)	Cómputo de despliegue.

5. Backend: estructura del código

```

backend/app/
+-- main.py      Punto de entrada FastAPI; registra routers y sirve el frontend.
+-- config.py    Variables de entorno y reglas de negocio (tasa 0.05%, límites KYC, TTL sesión).
+-- database.py  Motor SQLAlchemy y sesión (PostgreSQL o SQLite fallback).
+-- models.py    Modelos ORM (8 tablas) con tipo UUID portable.
+-- schemas.py   Contratos Pydantic (request/response).
+-- crypto.py    AES-256, enmascaramiento CLABE, validación módulo-10, hash encadenado, PBKDF2.
+-- security.py  JWT de sesión, 2FA TOTP, dependencia get_current_user.
+-- audit.py     Escritura de bitácora inmutable (hash SHA-256 encadenado).
+-- routers/     auth, kyc, vault, billing, checkout, membership, compliance.

```

Responsabilidades clave:

- `config.py` centraliza parámetros: `MEMBERSHIP_FEE_RATE = 0.0005`, `MAX_WHATSAPP_REMINDERS = 3`, `SESSION_TIMEOUT_MIN = 20`, `KYC_LIMITS_UDIS = {1:0, 2:3000, 3:8000, 4:None}`.
- `database.py` usa `check_same_thread` para SQLite y expone `get_db()` como dependencia.
- `main.py` ejecuta `create_all(checkfirst=True)` de forma idempotente (a prueba de carreras entre múltiples workers de Gunicorn) y monta el frontend estático.

6. Modelo de datos

Ocho tablas relacionales. Los campos sensibles se almacenan **cifrados con AES-256** (Regla 58a).

Tabla	Propósito	Campos sensibles / notas
users	Identidad de usuario	alias único con prefijo \$@; client_type (company/individual); kyc_level 1-4; totp_secret. Sin contraseña por ahora.
companies_metadata	Expediente de empresa	rfc_encrypted (AES-256), razón social, representante, CSF, logo.
individuals_metadata	Expediente de persona	curp_encrypted, full_name_encrypted (AES-256), rutas INE/liveness.
vault_accounts	Bóveda bancaria	clabe_encrypted (AES-256), clabe_masked, clabe_hash (SHA-256 anti-duplicados).
memberships	Saldo prepago	balance (Numeric 12,4), status (active/suspended/blocked).
charges	Cobros CoDi (Apéndice AD)	folio_codi único, monto, concepto (<=40), referencia (7 díg.), vigencia, pin_hash, status.
transactions	Liquidaciones inmutables	folio_codi, calculated_fee (0.05%), clave_rastreo SPEI, settled_at.
audit_logs	Bitácora inmutable	integrity_hash (SHA-256 encadenado), severity, event_category.

Portabilidad de UUID: el tipo GUID usa UUID nativo en PostgreSQL y CHAR(36) en

SQLite, permitiendo desarrollo local sin PostgreSQL.

Integridad: restricciones CHECK a nivel de base de datos (p. ej. balance >= 0,

alias LIKE '\$@%', client_type IN ('company', 'individual'), kyc_level BETWEEN 1 AND 4)

para reforzar reglas críticas incluso ante escritura directa.

El esquema PostgreSQL completo está en backend/init.sql; SQLAlchemy puede autogenerarlo

para desarrollo.

7. API REST

Base: /api/v1. Autenticación por **JWT Bearer** (excepto endpoints públicos de checkout).

Documentación interactiva en /docs (Swagger) y /redoc.

7.1 Módulo 1 - Autenticación y Onboarding (/auth)

Método	Ruta	Descripción
POST	/auth/register	Crea usuario por alias \$@... y tipo (Persona/Empresa), sin contraseña.

Método	Ruta	Descripción
POST	/auth/login	Inicia sesión por alias; devuelve JWT y totp_secret la primera vez.
GET	/auth/me	Perfil del usuario autenticado.
GET	/auth/users	Lista de alias registrados (para el selector de login).

7.2 Módulo 1 - KYC gradual (/kyc, Regla 72a)

Método	Ruta	Descripción
GET	/kyc/limits	Límites mensuales en UDIS por nivel.
POST	/kyc/upgrade	Sube a Nivel 2/3/4 validando los documentos requeridos por tipo.

7.3 Módulo 2 - Bóveda bancaria (/vault, Regla 58a)

Método	Ruta	Descripción
POST	/vault/accounts	Registra CLABE (validación módulo-10 + cifrado AES-256).
GET	/vault/accounts	Lista las cuentas de la bóveda (solo datos enmascarados).

7.4 Módulo 3 - Cobranza CoDi (/billing, Apéndice AD)

Método	Ruta	Descripción
POST	/billing/charges	Genera un cobro y su qr_payload (Mensaje de Cobro CoDi).
GET	/billing/charges	Lista los cobros; marca expirados.
POST	/billing/charges/{id}/remind	Envía recordatorio WhatsApp (máx. 3).

7.5 Módulo 4 - Pagos / Checkout (/checkout, Regla 71a)

Método	Ruta	Descripción
POST	/checkout/scan	Decodifica el contenido de un QR y devuelve el checkout.
GET	/checkout/{folio}	Vista pública del pagador (beneficiario enmascarado + CEP).
POST	/checkout/{folio}/pay	Liquida, genera clave de rastreo y deduce el 0.05%.

7.6 Módulo 5 - Membresía (/membership)

Método	Ruta	Descripción
GET	/membership	Saldo y estado de la membresía.
POST	/membership/recharge	Recarga de saldo prepago.

Método	Ruta	Descripción
POST	/membership/deduct	Deducción interna del 0.05% (402 si es insuficiente).

7.7 Módulo 6 - Auditoría y conciliación (/compliance, Regla 58a)

Método	Ruta	Descripción
GET	/compliance/audit-logs	Consulta de bitácoras (filtrable por categoría).
GET	/compliance/audit-logs/verify	Recalcula y verifica la cadena de hashes.
GET	/compliance/reconciliation	Transacciones con vínculo al CEP de Banxico.

Además: GET /api/health para health checks.

7.8 Ejemplo - generación de cobro

```
POST /api/v1/billing/charges
Authorization: Bearer <jwt>
Content-Type: application/json

{ "destination_account_id": "acc_...", "amount": 2500.00,
  "payment_concept": "Servicios profesionales", "numeric_reference": 1234567,
  "requires_pin": true, "pin": "1234" }
```

Respuesta 201: incluye folio_codi, payment_url y qr_payload (JSON del Mensaje de Cobro).

8. Seguridad y criptografía

Control	Implementación	Norma
Cifrado en reposo	AES-256 (CBC + PKCS7) para CLABE, RFC, CURP, nombres.	Regla 58a
Enmascaramiento en pantalla	CLABE (- - - - . . . 4567) y beneficiario por iniciales.	Regla 71a / Dicho
Hash de contraseña	PBKDF2-HMAC-SHA256 (200k iteraciones) para PINs.	Buenas prácticas
Anti-duplicados sin descifrar	clabe_hash = SHA-256 determinista.	-
Autenticación de sesión	JWT firmado (HS256), expiración configurable (20 min).	Regla 71a
Doble factor (2FA)	TOTP (RFC 6238) por usuario.	Apéndice P
Validación estructural CLABE	Dígito de control por doble módulo 10.	Sección 6 Manual SPEI
Bitácora inmutable	Cadena de hashes SHA-256 (h_n = SHA256(h_{n-1} + evento)).	Regla 58a
Canal cifrado	HTTPS/TLS terminado en Nginx; comunicación privada al backend.	Regla 58a / Apéndice AN
Cabeceras de seguridad	X-Content-Type-Options, X-Frame-Options, Referrer-Policy en Nginx.	Buenas prácticas

Verificación de integridad: `/compliance/audit-logs/verify` recalcula toda la cadena y reporta si fue alterada, indicando el primer registro roto.

Gestión de secretos: en producción, `AES_SECRET_KEY` y `JWT_SECRET` provienen de variables de entorno; se recomienda migrar la llave AES a un HSM/KMS (p. ej. AWS KMS).

9. Frontend (SPA)

9.1 Características

- **Multiusuario con login sin contraseña:** cada usuario elige tipo (Persona/Empresa) y un **alias único que inicia con \$@** (sugerible desde celular o alias de correo).
- **Seis módulos** navegables: Dashboard, Onboarding/KYC, Bóveda, Cobros CoDi, Pagar (lector de QR), Membresía y Bitácoras.
- **Lógica real en el cliente:** validación CLABE módulo-10, niveles KYC con campos por tipo, cálculo del 0.05%, bloqueo por saldo, y cadena de auditoría SHA-256 verificable.
- **Generación y lectura de QR:** el cobro produce un QR con el Mensaje de Cobro; el módulo Pagar lo lee por cámara, imagen, pegado o simulación.
- **Persistencia:** el estado se guarda en `localStorage` (clave `supercclabe_db_v2`) para pruebas; incluye botón de reinicio.

9.2 Modelo de datos en cliente

```
DB = { users: { "$@alias": { alias, client_type, kyc_level, profile, membership, vault[] } },
  charges: [ { owner, folio, amount, status, ... } ],
  transactions: [ { payee, folio, fee, clave, ... } ],
  audit: [ { ts, operator, action, hash } ],
  session: "$@alias" }
```

Los cobros guardan su owner; al pagar se deduce el 0.05% de la membresía del **beneficiario** (no del pagador), y la bitácora registra el alias como operador.

9.3 Integración con la API

`frontend/config.js` expone `API_BASE = "/api/v1"` y un flag `USE_API`. En el despliegue con Nginx, la SPA y la API comparten origen; el flag habilita el uso del backend real.

10. Flujo operativo clave - Cobro y pago vía QR (CoDi)

Beneficiario (comercio)	Pagador
-----	-----
1. Genera cobro (monto, concepto, referencia, vigencia, PIN opcional)	
2. Sistema crea folio + QR (Mensaje de Cobro CoDi, Apéndice AD)	-----> 3. Escanea el QR (cámara/imagen)
	4. Ve monto, concepto, beneficiario enmascarado y folio; captura PIN/2FA
	5. Acepta y paga
6. Liquidación SPEI/CoDi (<=4s):	
- estado -> PROCESSED	
- clave de rastreo generada	
- deducción 0.05% al beneficiario	
- registro en bitácora inmutable	
7. Comprobante con vínculo al CEP	----- (Banxico)

Si el saldo de membresía del beneficiario es insuficiente, la operación se bloquea (402) y se registra un evento FEE_DEDUCTION_BLOCKED.

11. Despliegue

11.1 Entornos

Entorno	Orquestación	Servidor app	Base de datos
Desarrollo local (sin Docker)	-	uvicorn --reload	SQLite (fallback)
Desarrollo (Docker)	docker-compose.yml	uvicorn	PostgreSQL 16
Producción (EC2)	docker-compose.prod.yml	Gunicorn + Uvicorn workers	PostgreSQL 16

11.2 Producción en EC2 (resumen)

1. Lanzar instancia Ubuntu 22.04 (t3.small+), Security Group con 22/80/443.
2. Instalar Docker con `deploy/ec2-user-data.sh`.
3. Copiar el proyecto a `/opt/supercalabe` (git o scp).
4. `cp .env.example .env` y definir secretos (`make secrets`).
5. `docker compose -f docker-compose.prod.yml up -d --build` (o `make up`).
6. Verificar `http://<IP>/api/health`; app en `/`, Swagger en `/docs`.

La guía detallada (HTTPS, dominio, respaldos, troubleshooting) está en `deploy/DEPLOY-AWS-EC2.md`.

11.3 Producción - características

- `restart`: `always` y `healthcheck` en cada servicio.
- Gunicorn con `WEB_CONCURRENCY` workers UvicornWorker; `max_requests` con `jitter`.
- Volúmenes persistentes `postgres_data` y `redis_data`.
- Nginx con `client_max_body_size 15m` y cabeceras de seguridad.

12. Configuración (variables de entorno)

Variable	Descripción	Ejemplo
DATABASE_URL	Cadena de conexión (PostgreSQL o SQLite).	postgresql://user:pass@db:5432/webapp_spei
AES_SECRET_KEY	Llave AES-256 (64 hex).	openssl rand -hex 32
JWT_SECRET	Secreto de firma de JWT.	openssl rand -hex 32
JWT_EXPIRE_MIN	Expiración de sesión (min).	20
REDIS_URL	Conexión a Redis.	redis://redis:6379/0
WEB_CONCURRENCY	Workers de Unicorn.	4
POSTGRES_USER/PASSWORD/DB	Credenciales de PostgreSQL.	-

13. Operación y mantenimiento

Acción	Comando
Iniciar / reconstruir	make up
Detener	make down
Ver logs	make logs
Estado	make ps
Generar secretos	make secrets
Respaldo de BD	make backup

Retención de bitácoras exigida: **>=6 meses** (aplicativo SPEI) y **>=1 año** (Canales

Electrónicos). Recomendado: snapshots EBS y pg_dump periódico a S3.

14. Mapeo normativo (resumen)

Módulo / control	Referencia
KYC gradual (niveles 1-4, límites UDIS)	Regla 72a
Cifrado AES-256, bitácoras, retención	Regla 58a
2FA, enmascaramiento, timeout de sesión	Regla 71a / Circular 9/2026
Mensajes de Cobro CoDi	Apéndice AD (Circular 12/2019)
Vínculo al CEP	Apéndice E
Conexión como PSPI / participante indirecto	Apéndice AN (Circular 14/2017)
Exclusión IFPE / membresía 0.05%	Ley Fintech / LTOSF
Certificación en Sandbox y auditoría externa	Apéndice O / Regla 74a

15. Limitaciones y roadmap técnico

Estado actual (base técnica / demostración):

- Autenticación sin contraseña (solo alias) - pensada para pruebas.
- El frontend opera como SPA con persistencia local; la API está disponible pero el cableado completo cliente<->servidor de todos los módulos es opcional (USE_API).
- Integraciones externas (RENAPO, INE, e.firma SAT, Dimo, banco patrocinador, WhatsApp) están **simuladas**.

Roadmap sugerido:

1. Cablear la SPA a la API real (login, cobros y pagos server-backed) para compartir datos entre dispositivos.
2. Contraseña + 2FA obligatorio y gestión de roles (Oficial de Seguridad, auditor).
3. Migrar la llave AES a AWS KMS/HSM y rotación de llaves.
4. HTTPS con certificado gestionado y dominio; WAF.
5. Integración real con el participante patrocinador y certificación en el Sandbox de Banxico (Apéndice O) y auditoría externa (Regla 74a).
6. Observabilidad: métricas, trazas y alertas; rate-limiting con Redis.

16. Glosario

Término	Significado
SPEI	Sistema de Pagos Electrónicos Interbancarios.
CoDi	Cobro Digital (esquema de mensajes de cobro de Banxico).
Dimo	Dinero Móvil (asociación celular-CLABE).
CLABE	Clave Bancaria Estandarizada (18 dígitos).
CEP	Comprobante Electrónico de Pago de Banxico.
IFPE	Institución de Fondos de Pago Electrónico.
PSPI	Proveedor de Servicios de Participación Indirecta.
KYC / PLD-FT	Conoce a tu cliente / Prevención de Lavado de Dinero y Financiamiento al Terrorismo.
UDIS	Unidades de Inversión (unidad de referencia indexada).
TOTP	Contraseña de un solo uso basada en tiempo (2FA).